

คู่มือโปรเจกต์ · V2.0 · 18 ก.ค. 2026

Ai102

โปรแกรมสร้างงานนำเสนอด้วย AI แบบ local-first ที่ทั้งร่างเนื้อหา ออกแบบ **ตรวจจรรีว** และแก้ไขสไลด์ **อัตโนมัติ** — รันได้ฟรีด้วยโมเดลโอเพนซอร์ส พร้อมระบบล็อกอิน OAuth จริง

Repository	github.com/Suppakris/Ai102
เว็บไซต์ที่ deploy	ai102-psi.vercel.app
พัฒนาต่อจาก	presentation-ai ของ ALLWEONE (fork ปรับแต่งสำหรับงานเรียน)
License	MIT
Framework	Next.js 16 · React 19 · TypeScript
แทนที่	คู่มือ V1.0 (16 ก.ค. 2026) — สิ่งที่เปลี่ยนไปทั้งหมดอยู่ในหัวข้อ 2

สารบัญ

1. ภาพรวมโปรเจกต์
2. สิ่งที่เปลี่ยนไปจากคู่มือ V1.0
3. ความแตกต่างจากโปรเจกต์ต้นทาง (Upstream)
4. ฟิเจอร์
5. เทคโนโลยีที่ใช้ (Tech Stack)
6. เริ่มต้นใช้งาน
7. ตัวแปรสภาพแวดล้อม (Environment Variables)
8. การตั้งค่าฐานข้อมูล (SQL Migrations)
9. การรันบน Docker
10. การทำงานเป็นทีม
11. วิธีใช้งาน
12. โครงสร้างโปรเจกต์
13. ปัญหาที่ทราบแล้ว
14. การมีส่วนร่วมพัฒนา
15. License

1. ภาพรวมโปรเจกต์

Ai102 คือโปรแกรมสร้างงานนำเสนอด้วย AI แบบ local-first ปรับแต่งจากโปรเจกต์โอเพนซอร์ส presentation-ai ของ ALLWEONE โดยคำเริ่มต้นการสร้างข้อความทำงานบนโมเดล **Ollama** ในเครื่อง (หรือผ่าน tunnel) — ไม่ต้องจ่ายค่า LLM คลาวด์ — และมี OpenRouter เป็นตัวเลือกอัปเกรดแบบเสียเงิน (แอดมินเป็นผู้ออกค่าใช้จ่าย)

แอปรับหัวข้อ (หรือไฟล์เอกสารต้นทาง: PDF, Word, Excel, CSV, ไฟล์ข้อความ) แล้วสร้างโครงร่างที่แก้ไขได้ จากนั้นสร้างสไลด์เต็มชุดพร้อมรูปภาพ AI ฟรี อิมที่ปรับแต่งได้ ข้อความ rich text กราฟ และส่งออกเป็น PowerPoint

ฟีเจอร์เด่นที่ต่อยอดจากการสร้างสไลด์คือ **AI Deck Review**: โมเดลผู้ตรวจจะให้คะแนนความชัดเจน การออกแบบ และความถูกต้องของเนื้อหา ตรวจสอบข้อกล่าวอ้าง (claim) ทุกข้อ และกดปุ่มเดียวเพื่อแก้ไขอัตโนมัติ (**Auto-fix**) โดยเขียนสไลด์ที่มีปัญหาใหม่ (พร้อมปุ่ม Undo)

การล็อกอินเป็น OAuth จริง (GitHub จำเป็น; Google และ Discord เป็นตัวเลือกเสริม) ฟีเจอร์หลักทั้งหมดใช้ฟรีสำหรับผู้ที่ใช้ที่ล็อกอินทุกคน

2. สิ่งที่เปลี่ยนไปจากคู่มือ V1.0

คู่มือ V1.0 (เขียนเมื่อ 16 ก.ค. 2026) บันทึกสถานะโค้ดรุ่นก่อนหน้านี้ ซึ่งตอนนี้เปลี่ยนไปมากแล้ว ถ้าเคยอ่าน V1.0 ให้ยกเลิกความเข้าใจเหล่านี้:

คู่มือ V1.0 บอกว่า	ปัจจุบัน (V2.0)
ระบบล็อกอินถูกปิด ทุกคนเป็น demo admin	OAuth จริงผ่าน Auth.js/NextAuth v5: GitHub จำเป็น, Google/Discord เสริม ผู้ใช้ใหม่เริ่มที่ <code>role: USER</code> การให้สิทธิ์แอดมินทำผ่านฐานข้อมูล (หัวข้อ 7)
เส้นทาง OpenRouter ใน Docker เป็นโค้ดที่ตายแล้ว	ใช้งานได้จริงแล้ว ตั้ง <code>OPENROUTER_API_KEY</code> จะมีกลุ่มโมเดลคลาวด์ 6 ตัวปรากฏใน model picker ให้ผู้ใช้ทุกคน
การสร้างรูปใช้ FAL (เสียเงิน) เป็นหลัก	ฟรีเป็นค่าเริ่มต้น ผ่าน Pollinations.ai (ไม่ต้องมี API key) ส่วน FAL เป็นตัวเลือกเสริมที่จำกัดเฉพาะแอดมิน
Slide audit agent ยังไม่ถูกสร้าง (ไม่มี API route)	เปิดตัวแล้วในชื่อ AI Deck Review: <code>POST /api/presentation/review-deck</code> , ปุ่ม Review ในตัวแก้ไข, Auto-fix พร้อม Undo, ให้ feedback สองภาษา (EN/TH), พร้อมชุดทดสอบ <code>pnpm review:test</code> และ <code>pnpm review:bash</code>
แนบไฟล์ต้นทางได้เฉพาะ PDF	รองรับ PDF, Word (.docx), Excel (.xlsx/.xls), CSV และไฟล์ข้อความ/Markdown — แปลงในเบราว์เซอร์ทั้งหมด ไฟล์ไม่ถูกอัปโหลดขึ้นเซิร์ฟเวอร์
ไม่มีหน้าแรก เข้าแอปแล้วดึงเข้า /presentationทันที	มีหน้า Landing สำหรับผู้ที่ยังไม่ล็อกอิน พร้อมปุ่มสลับโหมดสว่าง/มืด เมนูนำทาง (แฮมเบอร์เกอร์) ที่มีปุ่มออกจากระบบ และเอกลักษณ์สีน้ำเงิน/ฟ้าใหม่ทั้งแอป
Migration มีแค่ <code>0_init</code>	มี 3 migrations โดย <code>20260718150000_add_tenancy_and_agent_threads</code> ปิดช่องโหว่ที่ตาราง multi-tenancy อยู่ใน schema แต่ไม่อยู่ใน migrations (ทำให้ Docker DB ใหม่ขาดตาราง)

3. ความแตกต่างจากโปรเจกต์ต้นทาง (Upstream)

- ระบบล็อกอินเป็น **OAuth** จริง (`src/backend/auth.ts`) GitHub จำเป็น; Google และ Discord เป็นตัวเลือก — ปุ่มล็อกอินของแต่ละ provider จะแสดงเมื่อตั้งค่า credentials แล้วเท่านั้น ทั้งหมดฟรี ผู้ใช้ใหม่เริ่มที่ `role: USER`

ทดสอบล็อกอินบนโดเมนจริงที่ **deploy** เท่านั้น ห้ามใช้ลิงก์ **Vercel PR preview** — OAuth app มี callback URL ได้เพียงหนึ่งเดียว (production) การล็อกอินจาก preview URL จะล้มเหลวด้วย `error=Configuration` เสมอ นี่คือการพฤติกรรมปกติ ไม่ใช่บั๊ก

- การสร้างข้อความใช้ **Ollama** เป็นค่าเริ่มต้น และมี **OpenRouter** เป็นตัวเลือกเสียเงิน (`src/lib/model-picker.ts`) — ตั้ง `OPENROUTER_API_KEY` เพื่อปลดล็อกโมเดลคลาวด์ 6 ตัว (เจ้าของ key จ่ายตามโทเคน)
- การสร้างรูปฟรีเป็นค่าเริ่มต้น — Pollinations.ai (ไม่ต้องมี key) ทุกจุดที่สร้างรูป; FAL (Flux) เป็นตัวเลือกเสียเงินเฉพาะแอดมิน
- โค้ดฝั่ง **backend** อยู่ใน `src/backend/` — db, auth, tenant, rate limiting, คิวรูปภาพ และ LangGraph agent (ดู ARCHITECTURE.md)

4. ฟีเจอร์

ฟีเจอร์หลัก

- สร้างโครงสร้างด้วย AI แล้วสร้างสไลด์เต็มชุด บนโมเดล Ollama ในเครื่อง
- ใช้เอกสารของคุณเป็นข้อมูลต้นทาง: ลากไฟล์ **PDF, Word (.docx), Excel (.xlsx/.xls), CSV** หรือไฟล์ **ข้อความ/Markdown** ลงช่องพิมพ์ — แปลงในเบราว์เซอร์ทั้งหมด (ไฟล์ไม่ถูกอัปโหลด) และตัดให้พอดีกับ context ของโมเดล
- แก้ไขโครงสร้างก่อนสร้างจริง · สร้างแบบเรียลไทม์ · บันทึกอัตโนมัติ
- แชนเจ agent ในตัวแก้ไข (LangGraph + หน่วยงานจำ Postgres): แก้เลย์เอาต์/พื้นหลัง เปลี่ยน/สร้างรูปใหม่ เพิ่ม/ลบสไลด์ เปลี่ยนธีม คั่นเว็บ (ต้องมี `TAVILY_API_KEY`)

AI Deck Review และ Auto-fix (ฟีเจอร์เด่น)

- ปุ่ม **Review** ในแถบด้านบนของตัวแก้ไข: ให้คะแนน 0–10 ด้านความชัดเจน การออกแบบ ความถูกต้องของเนื้อหา พร้อมผลตัดสิน ผ่าน / ต้องแก้ไข
- **Claim audit**: ข้อกล่าวอ้างเชิงข้อเท็จจริงทุกข้อถูกระบุเป็น Supported / Unsupported / Unverifiable — ระบบส่ง prompt และโครงสร้างของงานนำเสนอไปเป็นข้อมูลอ้างอิง ทำให้ตรวจสอบ claim ที่มาจากโครงสร้างได้จริง
- **Auto-fix**: เด็คที่ไม่ผ่านจะถูก AI แก้หนึ่งรอบ — claim ที่ไม่มีหลักฐานจะถูกลบหรือปรับให้อ่อนลง (ไม่มีการแต่งหลักฐานปลอมเด็ดขาด) แล้วตรวจซ้ำและนำผลไปใช้ในตัวแก้ไข พร้อมปุ่ม **Undo** หากแปลงผลลัพธ์กลับเป็นสไลด์ไม่ได้ เด็คเดิมจะไม่ถูกแตะต้องเลย
- Feedback เป็นภาษาเดียวกับเด็ค (เด็คภาษาไทยได้ feedback ภาษาไทย) เด็คที่ข้อมูลน้อยเกินไปจะได้คำถามขอข้อมูลเพิ่มเติมแทนคะแนนมั่ว ๆ
- ผัง backend: `reviewSlides()` / `reviewAndRevise()` ใน `src/backend/ai/reviewSlides.ts` ให้บริการผ่าน `POST /api/presentation/review-deck` (ตรวจ session + rate limit) ผลลัพธ์ JSON ถูกบังคับด้วย schema
- ชุดทดสอบ: `pnpm review:test` และ `pnpm review:bash` (เด็คทดสอบเชิงรุก)

การออกแบบและปรับแต่ง

- ธีมในตัวหลายแบบ สร้างธีมเอง นำเข้าธีม PPTX; การเลือกธีมไม่ทำให้สีพื้นหลังของหน้าเปลี่ยนอีกต่อไป

เครื่องมือแนะนำ

- นำเสนอจากแอปโดยตรง · ลิงก์แชร์สาธารณะ · ส่งออก PPTX · กราฟ/อินโฟกราฟิก/สื่อฝัง · ตัวแก้ไข rich text (Plate) · คอมเมนต์ในสไลด์

รูปภาพ

- Pollinations.ai (ฟรี ค่าเริ่มต้น) · FAL Flux (เสียเงิน ตัวเลือก) · ภาพสต็อก Unsplash/Pixabay/Giphy · Google Custom Search

5. เทคโนโลยีที่ใช้ (Tech Stack)

หมวด	เทคโนโลยี
Framework	Next.js 16, React 19, TypeScript
Styling	Tailwind CSS v4 (โทนสีน้ำเงิน/ฟ้า รองรับโหมดสว่างและมืด)
ฐานข้อมูล	PostgreSQL (Supabase บน production, Docker Postgres ในเครื่อง) ผ่าน Prisma ORM + SQL migrations
ระบบล็อกอิน	Auth.js / NextAuth v5 — GitHub (จำเป็น), Google + Discord (ตัวเลือก)
สร้างข้อความ	Ollama (ค่าเริ่มต้น, native API) หรือ OpenRouter (ตัวเลือกเสียเงิน) ผ่าน LangChain + LangGraph
สร้างรูปภาพ	Pollinations.ai (ฟรี ค่าเริ่มต้น), FAL Flux (ตัวเลือกเสียเงิน)
คิวงาน	BullMQ + Redis (ตั้ง REDIS_URL เพื่อย้ายงานสร้างรูปออกจากเว็บโปเรซ)
UI / Editor	Radix UI · Plate Editor (platejs)
Lint/format	Biome (ตัวหลัก)

6. เริ่มต้นใช้งาน

สิ่งที่ต้องมี

- pnpm (ลือกที่ `pnpm@11.1.3`)
- ฐานข้อมูล PostgreSQL (Supabase, Neon หรือ Docker Postgres จากหัวข้อ 9)
- **Ollama** รันในเครื่องหรือเข้าถึงได้ผ่าน `OLLAMA_BASE_URL` พร้อมโมเดลอย่างน้อยหนึ่งตัว (เช่น `ollama pull qwen2.5:7b` — โมเดลอ้างอิงของพีเจอเรีว)
- GitHub OAuth app credentials (ฟรี) สำหรับล็อกอิน

ติดตั้ง

```
git clone https://github.com/Suppakris/Ai102.git
cd Ai102
pnpm install                # postinstall รัน prisma generate ให้
cp .env.example .env        # ใส่ DATABASE_URL + ตัวแปร auth (หัวข้อ 7)
pnpm db:migrate:deploy      # ติดตั้ง SQL migrations (หัวข้อ 8)
ollama pull qwen2.5:7b
pnpm dev
```

แอปรันที่ `http://localhost:3000` ผู้ที่ยังไม่ล็อกอินจะเห็นหน้า Landing; ล็อกอินด้วย GitHub (หรือ Google/Discord ถ้าตั้งค่าไว้) เพื่อเข้าแดชบอร์ด `/presentation`

7. ตัวแปรสภาพแวดล้อม (Environment Variables)

`.env.example` คือแหล่งอ้างอิงหลัก ตัวแปรสำคัญ:

ตัวแปร	จำเป็น?	หน้าที่
<code>DATABASE_URL</code>	จำเป็น	Connection string ของ Postgres สำหรับ Supabase ให้ใช้ Transaction pooler (พอร์ต 6543)
<code>NEXTAUTH_SECRET</code> / <code>NEXTAUTH_URL</code>	จำเป็น	เซ็นชื่อ session ของ Auth.js + URL หลักของแอป
<code>GITHUB_CLIENT_ID</code> / <code>GITHUB_CLIENT_SECRET</code>	จำเป็น	GitHub OAuth app (ฟรี) ขาดตัวแปร auth แล้วแอปจะไม่บูตบน production
<code>GOOGLE_CLIENT_ID</code> / <code>GOOGLE_CLIENT_SECRET</code>	ตัวเลือก	เพิ่มปุ่มล็อกอิน Google อย่าลืมเพิ่ม callback URL ในช่อง Authorized redirect URIs ของ Google Cloud Console
<code>DISCORD_CLIENT_ID</code> / <code>DISCORD_CLIENT_SECRET</code>	ตัวเลือก	เพิ่มปุ่มล็อกอิน Discord
<code>OLLAMA_BASE_URL</code>	ตัวเลือก	ซีไปยัง Ollama ระยะไกล/ผ่าน tunnel (เช่น ngrok) ระวัง: URL ของ ngrok เปลี่ยนทุกครั้งที่รีสตาร์ท — ต้องอัปเดตตัวแปรนี้ (แล้ว redeploy) เมื่อ tunnel รีสตาร์ท
<code>OLLAMA_DEFAULT_MODEL</code>	ตัวเลือก	โมเดลเริ่มต้น ควรตั้ง <code>qwen2.5:7b</code> บน production — โมเดลเล็กตัว fallback ให้ผลรวีวที่ใช้ไม่ได้ ต้องตั้งในที่ที่แอปรันจริง (เช่น Vercel) ไม่ใช่แค่ในเครื่อง
<code>OLLAMA_NUM_CTX</code>	ตัวเลือก	ขนาด context (ค่าเริ่มต้น 8192) — ค่า 2048 ของ Ollama เองเล็กเกินไปจน prompt ถูกตัด
<code>OPENROUTER_API_KEY</code>	ตัวเลือก	ปลดล็อกโมเดลคลาวด์ 6 ตัวให้ผู้ใช้ทุกคน เจ้าของ key จ่ายตามโทเคน
<code>FAL_API_KEY</code>	ตัวเลือก	อัปเกรดรูปภาพแบบเสียเงิน (เฉพาะแอดมิน) ไม่มี key ก็สร้างรูปฟรีผ่าน Pollinations.ai ได้
<code>REDIS_URL</code>	ตัวเลือก	ย้ายงานสร้างรูปไปที่ BullMQ worker; ไม่ตั้งก็รันแบบ inline
<code>TAVILY_API_KEY</code>	ตัวเลือก	เครื่องมือค้นเว็บของตัวสร้างโครงสร้างและแชท agent

การให้สิทธิ์แอดมิน

ผู้ใช้ใหม่เริ่มที่ `role: USER` การให้สิทธิ์แอดมิน (โมเดลรูป FAL แบบเสียเงิน, แก์อีมีระบบ):

```
pnpm db:studio    # เปิดตาราง User แล้วตั้ง role = ADMIN
-- หรือใช้ SQL:
UPDATE "User" SET role = 'ADMIN' WHERE email = 'their@email.com';
```

8. การตั้งค่าฐานข้อมูล (SQL Migrations)

Schema จัดการด้วย SQL migrations ที่ commit ไว้ใน `prisma/migrations/`:

```
pnpm db:migrate:deploy  # ติดตั้ง migrations ที่ค้างอยู่ทั้งหมด
pnpm db:migrate:dev      # หลังแก้ schema.prisma: สร้าง + ติดตั้ง migration ใหม่
pnpm db:studio           # เปิดดูฐานข้อมูล
```

Commit โฟลเดอร์ที่ถูกสร้างทุกครั้ง — ไฟล์ SQL นั้นคือตัวการเปลี่ยน schema ส่วน `pnpm db:push` มีไว้ทดสอบเล่นเท่านั้น ห้ามใช้กับฐานข้อมูลที่ใช้ร่วมกัน

Migrations ปัจจุบัน: `0_init` → `20260714110000_add_slide_audit` → `20260718150000_add_tenancy_and_agent_threads`

ฐานข้อมูลเดิมที่สร้างด้วย `db push` (รวมถึง Supabase production) มีตารางครบอยู่แล้ว ให้ baseline หนึ่งครั้งแทนการติดตั้งซ้ำ — ไม่งั้น `migrate deploy` จะพยายามสร้างตารางที่มีอยู่แล้ว:

```
pnpm exec prisma migrate resolve --applied 0_init
pnpm exec prisma migrate resolve --applied 20260714110000_add_slide_audit
pnpm exec prisma migrate resolve --applied 20260718150000_add_tenancy_and_agent_threads
```

หลัง baseline แล้ว `pnpm db:migrate:deploy` จะรายงาน "No pending migrations" และ migration ใหม่ ๆ ในอนาคตจะติดตั้งได้ตามปกติ

ไม่ต้อง seed — การลือกอินสร้าง record ผู้ใช้ให้อัตโนมัติผ่าน Prisma adapter

9. การรันบน Docker

ทั้งระบบ — front-end + back-end + Postgres + Redis + worker สร้างรูปภาพ — รันบน Docker โดยเครื่อง host ต้องมีแค่ Docker:

```
docker compose up --build
```

ลำดับการบูตจัดการให้อัตโนมัติ: Postgres พร้อม → คอนเทนเนอร์ `migrate` รัน `prisma migrate deploy` หนึ่งครั้ง → แอปเปิดที่ `http://localhost:3000` พร้อมคอนเทนเนอร์ worker (BullMQ) แยกต่างหากที่กินงานจากคิว Redis

การเข้าถึง LLM จากในคอนเทนเนอร์

- **Ollama บนเครื่อง host** (ค่าเริ่มต้น): compose ซี `OLLAMA_BASE_URL` ไป `host.docker.internal:11434`
- **OpenRouter**: ใส่ `OPENROUTER_API_KEY=sk-or-...` ในไฟล์ `.env` ข้าง `docker-compose.yml` (เส้นทางนี้ใช้งานได้จริงแล้ว — ค่าเตือน "โค้ดตาย" ของ V1.0 ไม่มีผลอีกต่อไป)

ระบบล็อกอินใน Docker

ตั้ง `GITHUB_CLIENT_ID` / `GITHUB_CLIENT_SECRET` จริง (และคู่ Google/Discord ถ้าต้องการ) ในไฟล์ `.env` เดียวกัน — ค่า placeholder ทำให้ระบบบูตและเปิดหน้าเว็บได้ แต่ล็อกอินไม่ได้

ข้อมูล Postgres เก็บใน volume `db-data`; คำสั่ง `docker compose down -v` จะลบทิ้ง

10. การทำงานเป็นทีม

	Track 1 — Prompt Testing	Track 2 — DevOps
ดูแล	Prompt templates, พฤติกรรม agent, การเลือกโมเดล/provider, คุณภาพผลลัพธ์	Docker, database migrations, ค่าคอนฟิก, การ deploy
ไฟล์	<code>src/backend/agent/**</code> , <code>src/backend/ai/**</code> , <code>src/lib/presentation/*prompt*</code> , <code>src/lib/model-picker.ts</code> , <code>src/app/api/presentation/**</code>	<code>Dockerfile</code> , <code>docker-compose.yml</code> , <code>prisma/migrations/**</code> , <code>prisma/schema.prisma</code> , <code>.env.example</code> , การตั้งค่า Vercel
วิธีทดสอบ	วัดคุณภาพด้วยชุดทดสอบที่มีจริง: <code>pnpm review:test</code> และ <code>pnpm review:bash</code> (Ollama ฟรี หรือ <code>--openrouter</code> เพื่อผลที่ทำซ้ำได้)	<code>docker compose up --build</code> ต้องบูตแอปที่ใช้งานได้จาก checkout ใหม่; การแก้ schema ต้องมาเป็น SQL migration ผ่าน <code>pnpm db:migrate:dev</code>
ขอบเขต PR	เฉพาะ prompt/agent	เฉพาะ infra

- การแก้ schema เป็นงานของ Track DevOps เสมอ แม้ PR ผัง prompt จะต้องใช้ — ให้แยก PR
- การแก้ prompt ตัดสินด้วยผลชุดทดสอบ ไม่ใช่การสุ่มดูผลครั้งเดียวที่บังเอิญสวย
- ทั้งสอง track แยก branch จาก `main` และ merge ผ่าน PR

11. วิธีใช้งาน

1. ล็อกอินที่เว็บไซต์ (หรือ `pnpm dev` ในเครื่อง) ด้วย GitHub, Google หรือ Discord
2. สร้างเด็กจากแดชบอร์ด: พิมพ์หัวข้อในช่อง prompt — หรือลากไฟล์ PDF/Word/Excel/CSV/ข้อความมาเป็นข้อมูลต้นทาง — เลือกจำนวนสไลด์ ภาษา และธีม แล้วส่งสร้าง ตรวจสอบ/แก้ไขโครงสร้างก่อนสร้างสไลด์จริง
3. แก้ไขในตัวแก้ไข: เปลี่ยนธีม (ปุ่ม $\vee$ ข้างชื่อเรื่องเปิดเมนูของเอกสาร ส่วน $\equiv$ เปิดเมนูนำทางแอป) แก้ข้อความตรง ๆ หรือสั่งแชท agent
4. รีวิว: กดปุ่ม **Review** ที่แถบบน จะได้คะแนน feedback และ claim audit ถ้าเด็กไม่ผ่าน กด **Auto-fix deck** — AI จะเขียนสไลด์ที่มีปัญหาใหม่แล้วตรวจซ้ำ กด **Undo** ได้ถ้าไม่ชอบผลลัพธ์ หมายเหตุ: เด็กที่เนื้อหาแต่งงานขึ้นทั้งหมดอาจยังไม่ผ่านหลัง Auto-fix — AI ลบ claim ปลอมได้ แต่สร้างข้อมูลจริงแทนให้ไม่ได้
5. ส่งงาน: ส่งออก `.pptx` นำเสนองานเบราว์เซอร์ หรือแชร์ลิงก์สาธารณะ

ถ้าแถบบนแสดง **"Backend issue detected"** แปลว่าเข้าถึงเซิร์ฟเวอร์ Ollama ไม่ได้ — ส่วนใหญ่เกิดจาก ngrok tunnel รีสตาร์ทแล้ว URL เปลี่ยน ให้อัปเดต `OLLAMA_BASE_URL` เป็น URL ใหม่แล้ว redeploy

12. โครงสร้างโปรเจกต์

ดูรายละเอียดเต็มใน ARCHITECTURE.md แผนที่ย่อของ `src/`:

```
src/
├─ app/                               เส้นทาง (routes) ของ Next.js App Router
│  ├─ page.tsx                       หน้า landing (ยังไม่ล็อกอิน) / redirect เข้าแดชบอร์ด
│  ├─ presentation/                  แดชบอร์ด, ขั้นตอนสร้าง, หน้าระหว่างสร้าง, ตัวแก้ไข ([id]/)
│  ├─ share/                         หน้าดูอย่างเดียวสำหรับลิงก์แชร์
│  ├─ api/                           route handlers (รวมถึง presentation/review-deck)
│  └─ _actions/                      Server Actions → ส่งต่อไป src/backend/**
├─ components/notebook/              UI โครงร่างสไลด์, ชีม, ปลั๊กอินตัวแก้ไข
├─ components/presentation/          โครงแอป: แถบบน (เมนู/อวาตาร์/ปุ่มสลับชีม),
│                                     ปุ่มรีวิว, แถงแก้ไข, โหมดนำเสนอ
├─ backend/                          db.ts · auth.ts (NextAuth v5) · tenant.ts · rate-limit.ts
│  ├─ ai/reviewSlides.ts             เอนจินรีวิว + auto-fix
│  ├─ queue/                         คิวรูปภาพ BullMQ (Pollinations/FAL)
│  └─ agent/                         LangGraph agent + เครื่องมือแก้ไขสไลด์
└─ lib/                              model-picker.ts · presentation/ (ชีม, prompts,
                                      pdf-extract.ts, office-extract.ts) · observability/
```

13. ปัญหาที่ทราบแล้ว

- ระบบค้นหาเอกสาร/RAG มีโครงแต่ยังไม่ได้สร้าง — ยังไม่มี vector search ต่อกับ agent ("คุยกับเอกสารที่อัปเดต" ยังใช้ไม่ได้ ส่วนการแนบเอกสารเป็นข้อมูลต้นทางตอนสร้างได้ใช้ได้จริง — คนละเส้นทางกัน)
- ผู้ใช้ใหม่เริ่มที่ `role: USER` — โมเดลรูป FAL เสียเงินและการแก้ไขระบบถูกซ่อนจนกว่าจะได้สิทธิ์แอดมิน (หัวข้อ 7) ฟิเจอร์หลักใช้ได้ครบที่ role USER
- ประวัติการรีวิวยังไม่ถูกบันทึก — ผลรีวิวดูใน dialog เท่านั้น ระบบเก็บประวัติเป็นฟิเจอร์ถัดไปที่ตกลงกันแล้ว
- Ollama เซิร์ฟเวอร์เดียวรับงานทีละคิว — การสร้างพร้อมกันหลายคนจะต่อแถวกัน
- ไม่รองรับ .doc รุ่นเก่า (Word 97–2003) เป็นไฟล์ต้นทาง (รองรับ .docx เท่านั้น); PDF ที่สแกนมาต้องมีข้อความที่เลือกได้
- มีทั้งคอนฟิก Biome และ Prettier; Biome เป็นตัวหลัก

14. การมีส่วนร่วมพัฒนา

ขั้นตอน

1. Fork, clone, `pnpm install`, แยก branch (`feature/ชื่อฟีเจอร์`)
2. แก้โค้ด; รัน `pnpm type` และ `pnpm check` ก่อน push
3. เปิด PR พร้อมชื่อและคำอธิบายที่ชัดเจน; ตอบ feedback จากผู้รีวิว

คำแนะนำ commit

`feat:` ฟีเจอร์ใหม่ · `fix:` แก้บั๊ก · `docs:` เอกสาร · `style:` จัดรูปแบบ · `refactor:` ปรับโครงสร้าง · `chore:` งานเครื่องมือ

15. License

MIT License ลิขสิทธิ์ดั้งเดิมเป็นของหิมี ALLWEONE; fork นี้เป็นงานดัดแปลง (derivative work) จัดทำเพื่อการศึกษา